

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 4 实验名称 实验项目 4.4 DMA 引擎与中断处理

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构的截图及说明

本次实验基于五级流水线 RISC-V 处理器, 扩展实现了外部中断响应机制, 并设计了一套基于队列结构的 DMA 硬件引擎, 最终通过软硬件协同完成内存数据的并行搬移。以下分别从处理器中断扩展、DMA 硬件设计、配套软件实现与性能测试四个部分展开说明。

1. RISC-V 处理器中断支持设计

- (a) 整体设计思路本次实验将外部中断视为一种优先级高于普通分支的特殊跳转: 当中断使能打开且外部中断信号有效时, 立刻冲刷当前流水线, 将程序计数器强制跳转到固定中断入口地址 0x100, 同时自动屏蔽后续中断, 避免中断嵌套。该设计可以最大程度复用已有的分支冲刷与流水线控制逻辑, 减少对原有处理器数据通路的修改。
- (b) 中断译码与控制信号首先在译码阶段新增对系统指令的识别, 通过操作码、funct3 与 funct7 字段共同译码出 MRET 中断返回指令, 对应代码如下:

```
// 系统指令与MRET译码
wire is_SYSTEM = (opcode == 7'b1110011);
wire is_MRET = is_SYSTEM && (funct3 == 3'b000)
               && (funct7 == 7'b0011000) && (rs2 == 5'b00010);
```

- (c) 中断冲刷与跳转逻辑在取指级将中断冲刷信号与普通分支冲刷信号合并为统一的流水线冲刷信号, 跳转目标优先选择中断入口地址, 保证中断优先级高于普通分支:

```
// 中断触发条件: 中断使能、外部中断有效、无复位、无待处理的跳转重定向
assign intr_flush = intr_enable && intr && !rst && !redirect_pending;

// 统一跳转目标: 中断优先级 > 分支跳转
assign pipe_jump_target = intr_flush ? 32'h100 : ex_branch_target;
assign pipe_flush = branch_flush | intr_flush;
```

- (d) 上下文寄存器与中断使能控制新增两组特权寄存器：‘mepc’用于保存中断触发时的程序计数器，保证中断返回后能回到原程序断点继续执行；‘intr_enable’作为全局中断屏蔽位，进入中断时自动关闭，执行 MRET 指令时重新打开：

```
always @(posedge clk) begin
    if (rst) begin
        mepc <= 32'h0;
        intr_enable <= 1'b1; // 复位后默认开中断
    end else if (intr_flush) begin
        mepc <= pc_reg;      // 保存断点PC
        intr_enable <= 1'b0; // 进入中断，屏蔽后续中断
    end else if (idex_is_MRET && idex_valid) begin
        intr_enable <= 1'b1; // 执行MRET，恢复中断使能
    end
end
```

2. DMA 引擎硬件设计

- (a) 整体架构 DMA 引擎采用读写解耦的设计思路，整体由控制寄存器组、读引擎、写引擎和 FIFO 数据缓冲四部分构成，核心结构如图所示。读引擎负责从源内存读取数据并写入 FIFO，写引擎负责从 FIFO 读取数据并写入目的内存，二者通过 FIFO 实现速率匹配与解耦，可并行工作。

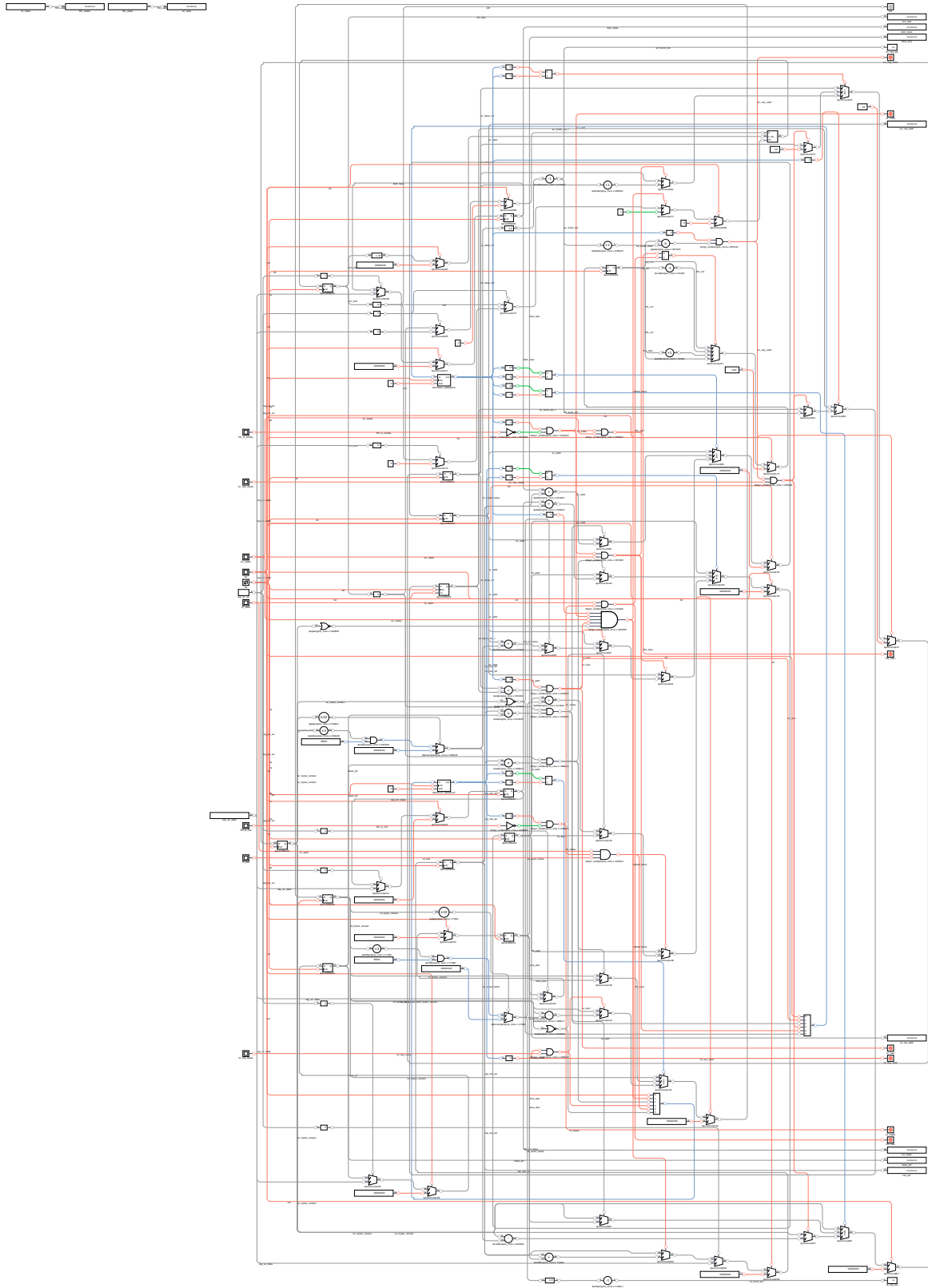


图 1: DMA 引擎核心模块整体结构框图

- (b) 控制与状态寄存器组 DMA 引擎对外暴露 6 组 32 位可访问寄存器, 处理器通过 MMIO 方式进行读写配置。寄存器写入采用独热码使能信号控制, 保证每次写操作仅更新一个寄存器, 避免多寄存器同时改写的竞争问题。各寄存器功能定义如下:

- src_base / dest_base: DMA 读、写操作的内存基地址
- head_ptr / tail_ptr: 任务队列的头、尾指针, 分别由处理器和 DMA 硬件更新
- dma_size: 单个子缓冲区的字节长度
- ctrl_stat: 控制状态寄存器, bit0 为 DMA 全局使能位, bit31 为中断标志位, 采用写 1 清零机制

其中控制状态寄存器的读写逻辑是中断交互的核心, 对应代码如下:

```
if (reg_wr_en[5]) begin
    reg_ctrl_stat[0] <= reg_wr_data[0]; // EN位普通读写
    // INTR位: 写1清零(W1C), 写0不改变原值
    if (reg_wr_data[31]) begin
        reg_ctrl_stat[31] <= 1'b0;
    end
end
// 硬件自动置位中断标志, 优先级高于软件清零
if (subbuf_done) begin
    reg_ctrl_stat[31] <= 1'b1;
end
assign intr = reg_ctrl_stat[31];
```

(c) 读引擎状态机设计读引擎采用四状态有限状态机, 实现子缓冲区的突发读取流程, 各状态定义如下:

- RD_IDLE: 空闲状态, 当 DMA 使能且队列非空时, 初始化地址与剩余字节数, 进入请求状态
- RD_REQ: 发送读突发请求, 等待内存控制器握手响应
- RD_DATA: 接收读回数据, 在 FIFO 未滿时写入缓冲, 直到本次突发传输结束
- RD_DONE: 整个子缓冲区读取完成, 等待写引擎处理完毕后回到空闲状态

状态机核心跳转逻辑代码片段:

```
case (rd_state)
    RD_IDLE: begin
        if (reg_ctrl_stat[0] && (reg_tail_ptr != reg_head_ptr)) begin
            rd_state <= RD_REQ;
            rd_addr <= reg_src_base + reg_tail_ptr;
            rd_bytes_remain <= reg_dma_size;
        end
    end
    RD_REQ: begin
        if (rd_req_valid && rd_req_ready) begin
            rd_state <= RD_DATA;
            rd_bytes_remain <= rd_bytes_remain - rd_burst_bytes;
        end
    end
    RD_DATA: begin
        if (rd_valid && rd_ready && rd_last) begin
            rd_state <= (rd_bytes_remain == 0) ? RD_DONE : RD_REQ;
            rd_addr <= rd_addr + rd_burst_bytes;
        end
    end
    RD_DONE: begin
```

```

        if (subbuf_done) rd_state <= RD_IDLE;
    end
endcase

```

(d) 写引擎状态机设计写引擎采用三状态有限状态机，与读引擎对称设计，额外增加了突发长度锁存与 FIFO 数据量判断，避免 FIFO 数据不足时发起写请求导致传输错误。各状态定义如下：

- WR_IDLE: 空闲状态，队列非空时初始化写地址与剩余字节数
- WR_REQ: 等待 FIFO 中数据量满足一个突发长度后，发送写突发请求
- WR_DATA: 从 FIFO 读取数据逐拍发送，直到本次突发传输结束

写请求与最后一拍标记的核心逻辑：

```

// 仅当FIFO数据量足够时才发起写请求
assign wr_req_valid = (wr_state == WR_REQ) && (fifo_cnt >= wr_burst_total);
// 锁存本次突发长度，避免传输中组合逻辑变化导致last错误
always @(posedge clk) begin
    if (wr_req_valid && wr_req_ready)
        wr_burst_len_r <= wr_burst_len;
end
// 最后一个数据标记
assign wr_last = (wr_state == WR_DATA) && (wr_data_cnt == wr_burst_len_r);

```

(e) Burst 传输与对齐机制单次突发传输最大长度为 32 字节（对应 8 个 32 位数据），对于子缓冲区末尾不足 32 字节的部分，向上按 4 字节对齐，保证所有内存访问都满足字对齐要求。突发长度计算逻辑如下：

```

wire [31:0] wr_burst_bytes = (wr_curr_rem >= 32) ? 32 : ((wr_curr_rem + 3) & ~32'd3);
wire [4:0] wr_burst_len = (wr_burst_bytes >> 2) - 1'b1;

```

(f) FIFO 数据计数与队列同步通过 ‘fifo_cnt’ 寄存器实时统计 FIFO 内的有效 32 位字数，为写引擎的突发请求发起提供判断依据。计数器采用读写使能的组合逻辑更新，位宽设置为 9 位，可支持最大 512 字的 FIFO 深度。

```

always @(posedge clk) begin
    if (rst) begin
        fifo_cnt <= 9'd0;
    end else begin
        case ({fifo_wen, fifo_rden})
            2'b10: fifo_cnt <= fifo_cnt + 1'b1;
            2'b01: fifo_cnt <= fifo_cnt - 1'b1;
            default: ;
        endcase
    end
end

```

任务队列采用经典的生产者-消费者模型：处理器作为生产者，填充子缓冲区后更新头指针；DMA 作为消费者，搬移完成后更新尾指针。当尾指针与头指针相等时表示队列为空，DMA 停止工作。

3. 配套软件设计

- (a) 启动代码与固定地址布局启动代码中通过‘.org’指令显式固定全局变量的内存地址,其中‘dma_buf_stat’严格固定在 0x10 地址,与中断服务程序的硬编码访问保持一致,避免地址偏移导致的计数错误。关键代码如下:

```
start:
    li    sp, 0x4000
    j     continued

.org 0x0c
global_result:
    .word 0xffffffff

.org 0x10
dma_buf_stat:
    .word 0x0

continued:
    la    sp, _SP_START_
    call  main
    call  hit_good_trap
```

- (b) 中断服务程序设计中断服务程序采用尾指针差值法计算完成的子缓冲区数量,保证即使出现中断合并也能准确计数。程序执行流程为:保存寄存器现场、清除 DMA 中断标志、计算并更新缓冲区计数、恢复现场、中断返回。核心代码片段如下:

```
intr_handler:
    addi  sp, sp, -32    # 开辟栈帧保存现场
    sw    t0, 0(sp)
    sw    t1, 4(sp)
    # ... 保存其余用到的寄存器

    # 清除中断标志: bit31写1清中断, bit0写1保使能
    li    t0, 0x60020000
    li    t1, 0x80000001
    sw    t1, 0x14(t0)

    # 计算尾指针增量, 循环减法求完成的缓冲区数
    lw    t1, 0x08(t0)   # 当前tail_ptr
    lw    t2, 0x10(t0)   # 单缓冲区大小
    la    t3, last_tail_ptr
    lw    t4, 0(t3)      # 上一次的尾指针
    sub   t4, t1, t4     # 尾指针增量
```

程序最后通过‘mret’指令返回,硬件自动恢复 PC 与中断使能。

- (c) 数据搬移主程序逻辑应用层采用生产者模式: CPU 循环向子缓冲区填充数据,每填完一个就更新 DMA 头指针并递增‘dma_buf_stat’计数;全部任务提交后,主程序轮询‘dma_buf_stat’直到其归零,代表所有子缓冲区都被 DMA 搬移完成。

4. 性能测试与结果分析

(a) 测试环境与条件测试在 FPGA 平台上运行,使用相同的 1MB 测试数据,分别采用纯软件内存拷贝与 DMA 硬件搬移两种方式,统计完整数据搬移的总耗时,并计算有效带宽与加速比。

(b) 性能对比结果两种搬移方式的测试数据如下表所示:

表 1: 不同搬移方式性能对比					
测试程序	搬移方式	总时间 (ms)	搬移数据量	有效带宽 (MB/s)	加速比
data_mover_no_dma	CPU 纯软件拷贝	1649.52	1 MB	0.61	1.00
data_mover_dma	DMA+ 中断协同	710.88	1 MB	1.41	2.32

(c) 结果分析 DMA 搬移方式相比纯软件拷贝获得了 2.32 倍的性能提升,其优势主要来源于两方面:一是数据搬移的核心工作完全由 DMA 硬件完成,CPU 仅负责任务配置与中断收尾,二者可以并行工作,CPU 可同时执行其他计算任务;二是 DMA 采用 Burst 突发传输,减少了总线握手的开销,提升了内存总线的有效利用率。

二、 实验过程中遇到的问题、对问题的思考过程及解决方法

本次实验调试过程中最核心的问题是 DMA 模式性能严重异常,以下详细说明问题的定位、分析与解决过程。

1. 问题现象初始版本的 DMA 测试中,程序虽然最终可以正常结束并通过校验,但执行时间高达约一百万毫秒,是纯软件模式的数百倍,完全违背了 DMA 加速的设计初衷。
2. 排查与定位过程
 - (a) 首先排查 DMA 硬件传输效率:通过波形观察读写引擎的状态跳转,确认 Burst 请求可以正常发起,FIFO 没有出现读写死锁,读写引擎也没有异常停转,排除了硬件传输本身的效率问题。
 - (b) 其次排查中断逻辑:确认中断信号可以正常触发,中断服务程序能够正确进入与返回,没有出现中断风暴或死循环的情况,排除了中断反复触发导致 CPU 卡死的可能。
 - (c) 最终定位到内存地址布局问题:检查启动代码 'start.S' 的变量地址分配,发现 'global_result' 被错误放置在 0x08 地址,导致紧随其后的 'dma_buf_stat' 实际地址偏离了预期的 0x10。
3. 根本原因中断服务程序中通过硬编码地址 0x10 来更新 'dma_buf_stat' 计数,但实际变量地址发生了偏移,导致计数更新写入了错误的内存位置。主程序轮询的 'dma_buf_stat' 变量始终不会递减,程序陷入长时间的空等待,最终表现为总耗时异常偏高。
4. 解决方案调整启动代码中的地址布局,将 'global_result' 固定在 0x0c 地址,保证 'dma_buf_stat' 严格位于 0x10 地址,与中断服务程序的访问地址完全匹配。
5. 修复效果修正地址布局后,DMA 模式的执行时间从异常的数百万毫秒降至 710.88ms,功能正常且性能符合预期,问题得到彻底解决。

三、 对讲义中思考题的理解和回答

1. 思考题 1:为什么中断服务程序中需要通过 tail 指针差值计算完成的缓冲区数量,而不是简单地每次中断计数减 1?

- (a) 应对中断合并场景: 当 DMA 传输速度较快、CPU 中断响应存在延迟时, 可能出现多个子缓冲区连续完成但只触发一次中断的情况。如果采用简单的“一次中断减一”逻辑, 会导致计数比实际完成数偏大, 主程序出现死等。差值计算方式通过尾指针的实际增量反推完成数量, 不受中断响应时机的影响。
- (b) 提升鲁棒性: 即使出现偶发的中断丢失, 差值法也能保证最终计数的准确性, 不会因为个别中断丢失导致整个流程卡死。
- (c) 符合工业设计规范: 该设计是实际 DMA 控制器的通用实现方式, 通过硬件指针而非中断计数来同步状态, 可靠性更高。

2. 思考题 2: DMA 中断标志位采用写 1 清零 (W1C) 的设计有什么优势?

- (a) 操作原子性: 单次写操作即可完成清零, 无需“读-改-写”三步操作, 避免了读写间隙硬件再次置位标志导致的竞争与冒险问题, 时序更安全。
- (b) 功能安全性: 软件仅能通过写 1 清除标志, 无法误写将标志位置 1, 保证中断标志只能由硬件置位, 符合外设状态寄存器的设计规范。
- (c) 实现简洁: 硬件仅需判断写入数据的对应比特, 无需复杂的读写合并逻辑, 资源开销小且不易出错。

四、 实验所耗时间

在课后, 你花费了大约 4 小时完成此次实验。